



SIEMENS EDA

Hackathon Lab Guide

Release V2026.2+

SIEMENS

Unpublished work. © 2025 Siemens

This material contains trade secrets or otherwise confidential information owned by Siemens Industry Software, Inc., its subsidiaries or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this information is strictly limited as set forth in Customer's applicable agreement with Siemens. This material may not be copied, distributed, or otherwise disclosed outside of Customer's facilities without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This document is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made. Siemens disclaims all warranties with respect to this document including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement of intellectual property.

The terms and conditions governing the sale and licensing of Siemens products are set forth in written agreements between Siemens and its customers. Siemens' **End User License Agreement** may be viewed at:
www.plm.automation.siemens.com/global/en/legal/online-terms/index.html.

No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

TRADEMARKS: The trademarks, logos, and service marks ("Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third-party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' trademarks may be viewed at:
www.plm.automation.siemens.com/global/en/legal/trademarks.html. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

Support Center: support.sw.siemens.com
Send Feedback on Documentation: support.sw.siemens.com/doc_feedback_form

Table of Contents

Overview	4
FPGA Hackathon Flow in Questa Developer	4
Configurator launch and VIP integration	10
Debugging Protocol Failures using Questa One Sim Debug	14

Overview

Use this section as a step-by-step interactive guide to move from the base Vivado project into Questa Developer, edit the missing RTL, compile changes immediately, and run simulation. Each step below focuses on the exact actions to take in the tool so you can follow the flow directly while working.

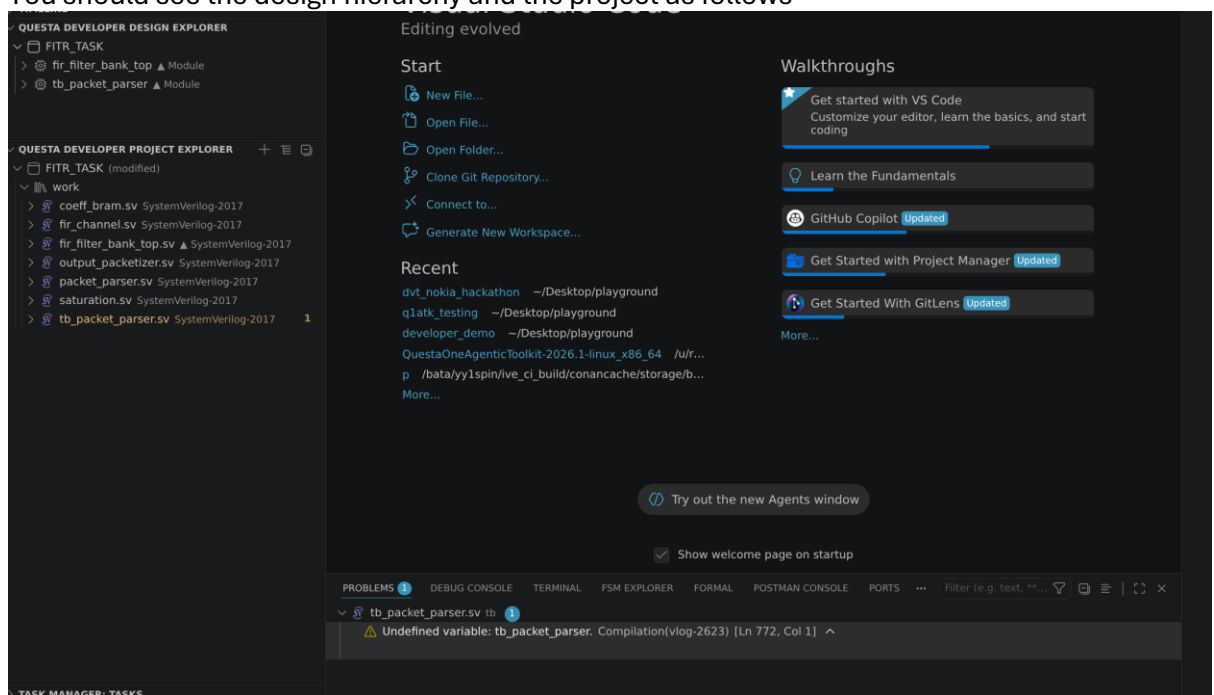
The intended workflow is simple: open and verify the project in Vivado, launch the simulator flow into Questa Developer, edit only the required source file, compile after every small change, and use simulation results to guide the next fix. Follow the steps in order and use the screenshot placeholders to insert matching tool views.

[Screenshot placeholder: High-level flow diagram showing Vivado project setup, import into Questa Developer, code editing, on-the-fly compilation, and simulation run]

FPGA Hackathon Flow in Questa Developer

Develop in VS Code with Questa Developer.

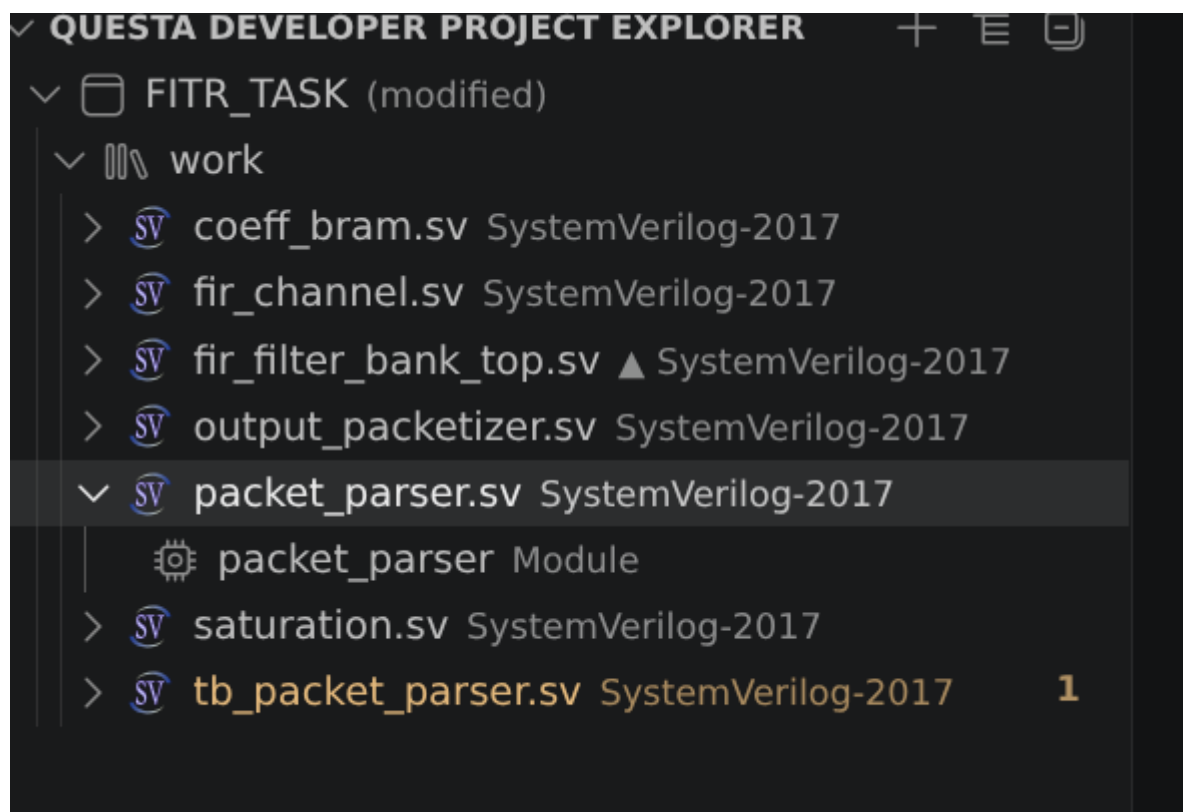
1. In the terminal, open VS Code
%> code
2. It will open VS Code with everything ready to start developing your RTL
3. You should see the design hierarchy and the project as follows



Expected result: The project is open in Questa Developer and ready for source editing and compile and it will be auto compiling while editing

Start writing the missing part in the packet parser.

1. In Questa Developer, open “packet_parser” file you identified



2. Read the module declaration and check the ports, parameters, and existing comments before editing.

3. Move to the section where the missing logic should be added. The section will be surrounded by TODO 1 and TODO2.

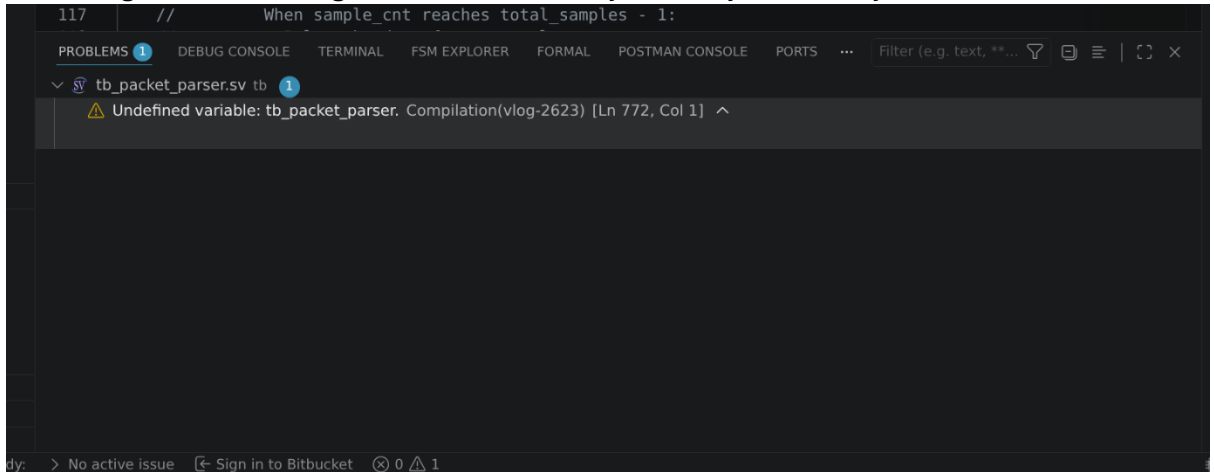
```

46  rtl> packet_parser.v
79  ): // s_ready is always asserted
80  assign s_ready = 1'b1;
81
82  // =====
83  // TODO 1: Main sequential logic – FSM state transitions + header/counter logic
84  // =====
85  // Implement an always_ff block (posedge clk or negedge rst_n) that:
86  //
87  // RESET (rst_n == 0):
88  //   - Set state to st_header
89  //   - Clear all registered header fields (r_num_ch, r_num_taps, r_num_samp)
90  //   - Clear all counters (coeff_cnt, sample_cnt, total_coeffs, total_samples)
91  //   - Clear coefficient addressing (coeff_ch_idx, coeff_tap_idx)
92  //   - Deassert hdr_valid and pkt_done
93  //
94  // NORMAL OPERATION:
95  //   - Default hdr_valid and pkt_done to 0 each cycle (pulse behavior)
96  //   - Implement a case statement on 'state' with three states:
97  //
98  // st_header:
99  //   When s_valid, extract header fields from s_data:
100 //     s_data[31:24] → r_num_ch   (number of channels, N)
101 //     s_data[23:16] → r_num_taps (number of taps, T)
102 //     s_data[15:0]  → r_num_samp (number of samples, S)
103 //   Compute total_coeffs = N * T, total_samples = N * S
104 //   Clear coeff_cnt, sample_cnt, coeff_ch_idx, coeff_tap_idx
105 //   Pulse hdr_valid for one cycle
106 //   Transition to ST_COEFFS
107 //
108 // ST_COEFFS:
109 //   When s_valid, increment coeff_cnt
110 //   Track per-channel tap index for BRAM addressing:
111 //     If coeff_tap_idx == r_num_taps - 1: reset tap index, advance channel
112 //     Else: increment coeff_tap_idx
113 //   When coeff_cnt reaches total_coeffs - 1: transition to ST_SAMPLES
114 //
115 // ST_SAMPLES:
116 //   When s_valid, increment sample_cnt
117 //   When sample_cnt reaches total_samples - 1:
118
119 logic [0:0] samp_in_ch; // sample index within current channel
120
121 // =====
122 // TODO 2: Sample channel tracking – per-channel sample counter
123 // =====
124 //
125 // Implement an always_ff block (posedge clk or negedge rst_n) that:
126 //
127 // RESET (rst_n == 0):
128 //   - Clear ch_idx and samp_in_ch to 0
129 //
130 // NORMAL OPERATION:
131 //   - When in st_header state AND s_valid:
132 //     Reset ch_idx and samp_in_ch to 0 (new packet starting)
133 //   - When in ST_SAMPLES state AND s_valid:
134 //     If samp_in_ch reaches r_num_samp - 1:
135 //       Reset samp_in_ch to 0 and advance ch_idx (next channel)
136 //     Else:
137 //       Increment samp_in_ch
138 //
139 // =====

```

4. Write the required RTL only in the intended implementation area as stated in the section and refer to <link for packet parser specs>
5. Do NOT rename the file, change the interface, or modify unrelated files unless the workflow explicitly requires it.
6. You will see any syntax error coming automatically in the VS Code “Problems” window.

You can ignore the warnings but make sure that you have your code syntax error clean



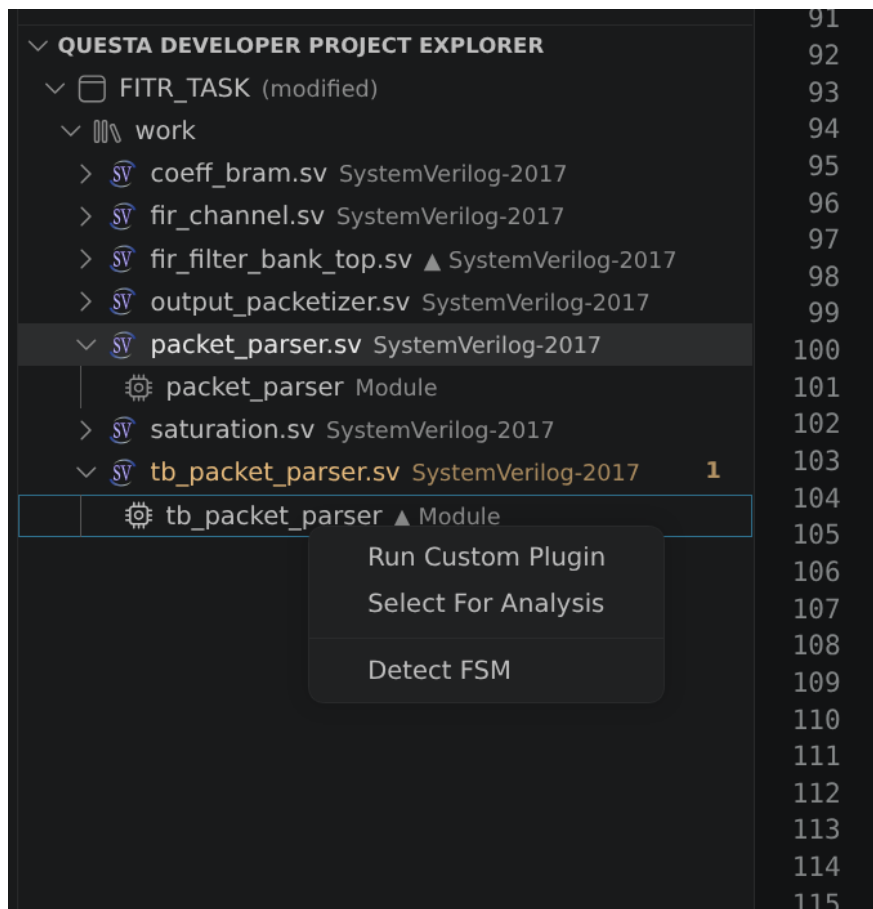
to be able to run simulation.

7. Save the file after each meaningful code update.

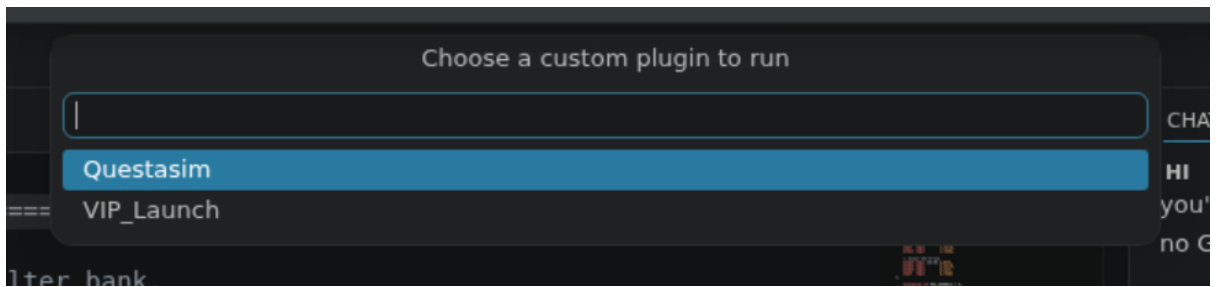
Expected result: The missing logic is added to the correct file and the source is ready to compile.

Run simulation.

1. Start simulation from the Questa Developer Design Explorer. RMB on the TB and choose “Run Custom Plugin”



2. Click on “QuestaSim” from the drop-down menu



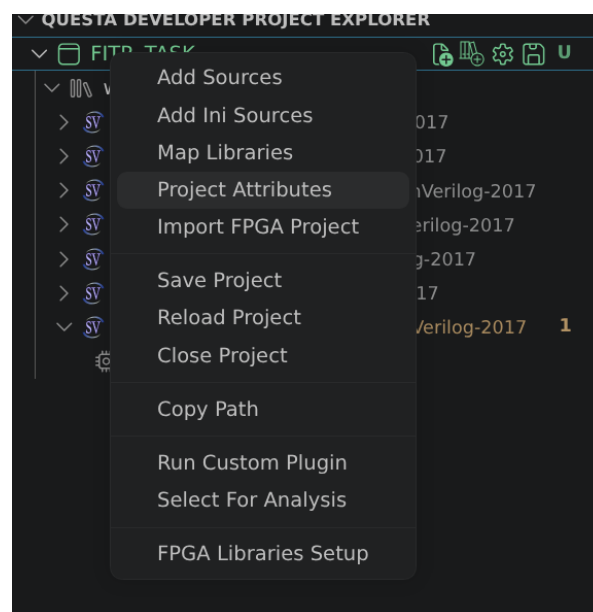
3. Wait for elaboration and simulation startup to complete.
4. Open the transcript and check if your simulation pass.

```

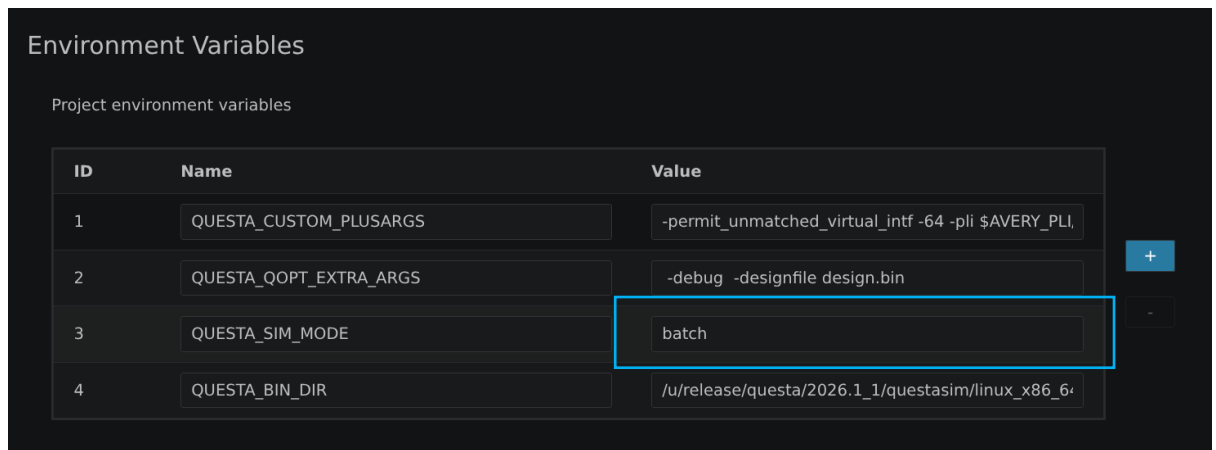
PROBLEMS 1 DEBUG CONSOLE TERMINAL FSM EXPLORER FORMAL POSTMAN CONSOLE PORTS OUTPUT GITLENS
# MISMATCH BRAM[59]: expected 0x0bc3 got 0x0000
# MISMATCH BRAM[60]: expected 0x0bc4 got 0x0000
# MISMATCH BRAM[61]: expected 0x0bc5 got 0x0000
# MISMATCH BRAM[62]: expected 0x0bc6 got 0x0000
# MISMATCH BRAM[63]: expected 0x0bc7 got 0x0000
# FAIL : TC14: all 64 BRAM cells match ch*1000+tap pattern
# FAIL : TC14: pkt done fired
# FAIL : TC14: coeff we = 64
# FAIL : TC14: samples = N*S = 32
#
=====
# Test Summary : 8 PASSED, 97 FAILED
#
# RESULT : FAILURES DETECTED - review output above
# ** Note: $finish : /home/kimagdy/Desktop/playground/dvt_nokia_hackathon/hackathon_exercise/tb/
# tb_packet_parser.sv(755)
# Time: 10085 ns Iteration: 1 Instance: /tb packet parser
# End time: 01:20:17 on Jun 02,2026, Elapsed time: 0:00:04

```

5. To debug your code in QuestaSim, RMB on the project and click on “Project Attributes”



Change simulation mode from “batch” to “gui”



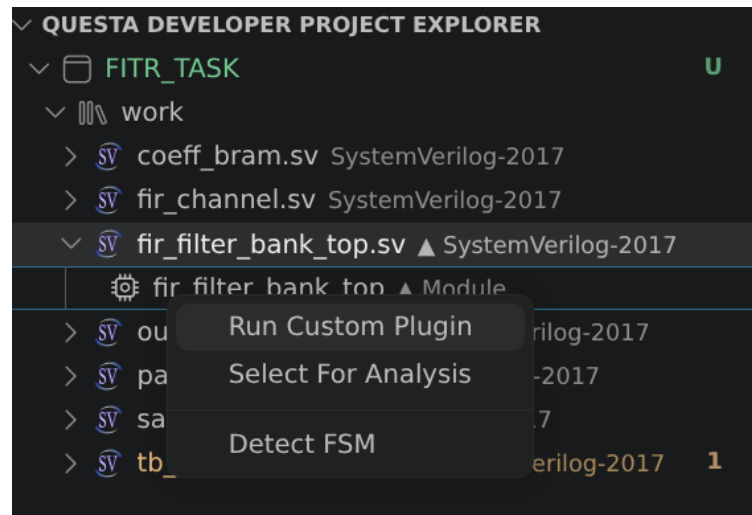
6. If the result is incorrect, go back to the source file, update the code, compile again, and rerun the simulation.

Expected result: The simulation runs successfully and confirms the implemented logic is functioning correctly. [You get the full mark if you pass this RTL. The upcoming tasks are for bonus.](#)

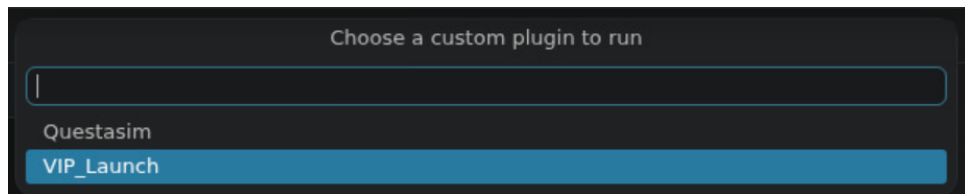
Configurator launch and VIP integration

After your design passes the normal testbench, Here comes the next step which is integrating your design with the VIP interface using configurator to generate a UVM testbench and run the UVM environment with attaching the scoreboard.

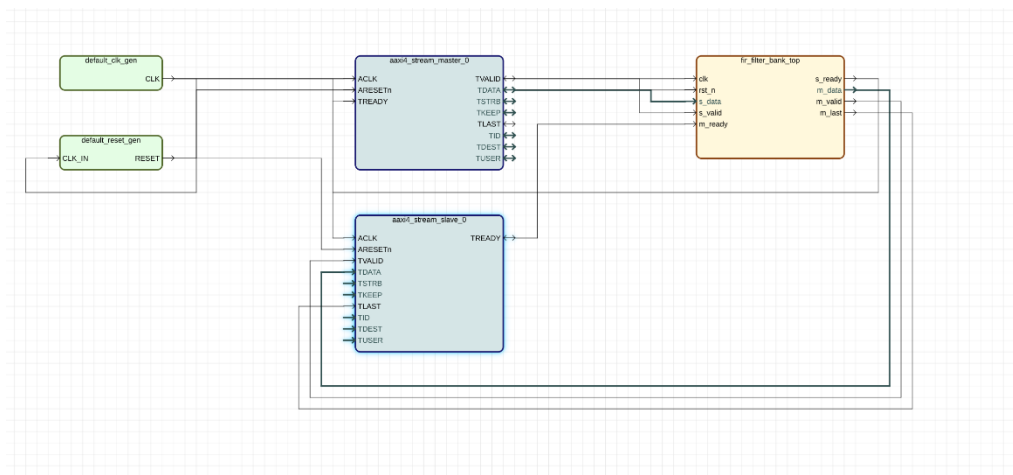
1. Start simulation from the Questa Developer Design Explorer. RMB on the TB and choose **“Run Custom Plugin”**



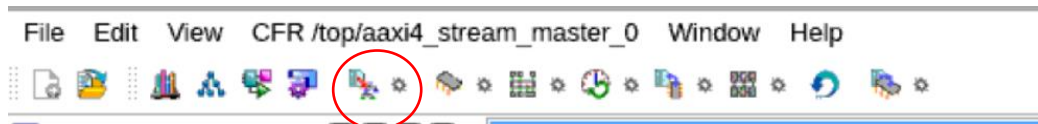
2. Click on **“VIP_Launch”** from the drop-down menu



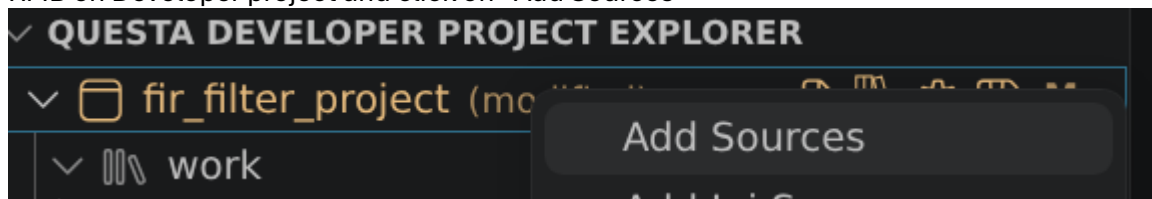
3. You will find your DUT connected to the VIP interface



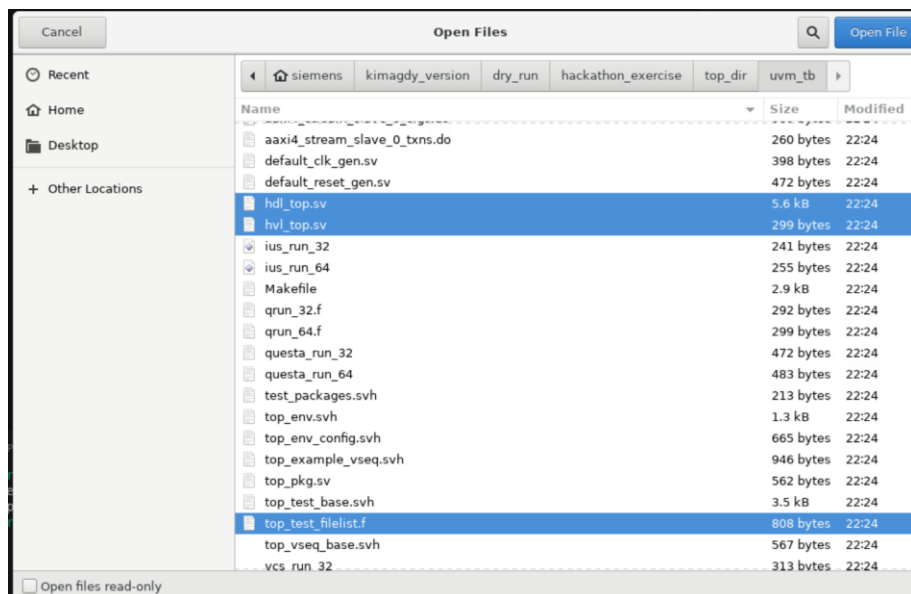
4. Then Press on generate TB to generate your uvm_tb



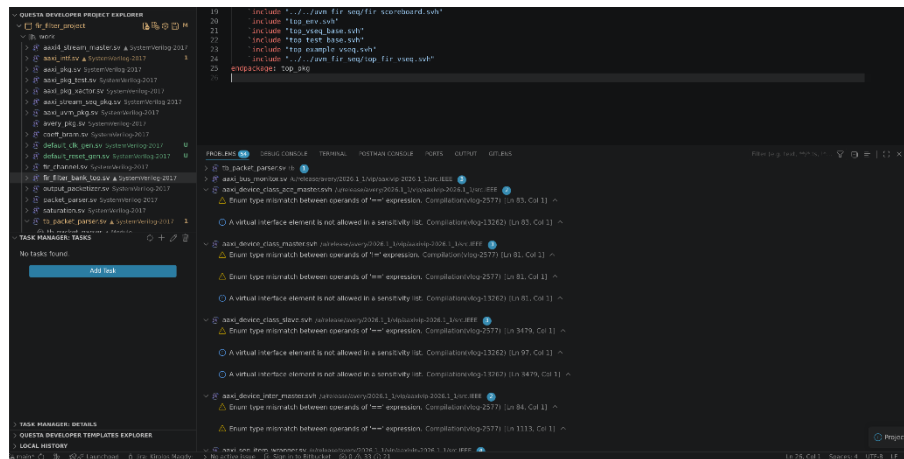
5. You will find the auto generated directory “top_dir” generated after launching configurator
6. Close configurator and go back to VS Code
7. Execute the following commands to add the sequences and the rest of the IP-Level TB in the <task_directory>:
`%> cp ip_tb_ready/* ./top_dir/uvm_tb/`
8. Add the IP level generated TB to Developer to connect it with your design
 RMB on Developer project and click on “Add Sources”



Navigate to the uvm_tb directory and choose the top_test_filelist.f , hdl_top.sv, hvl_top.sv

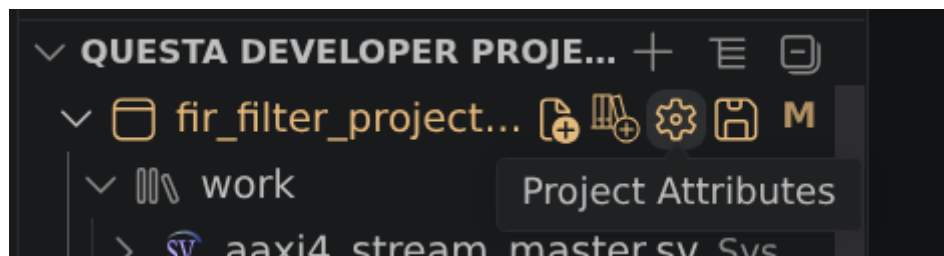


9. Your project should be compilation error free and looks like the screenshot

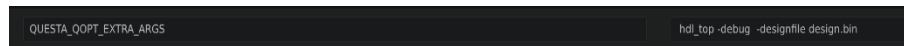


10. Now edit the project attributes to add the top module “hdl_top”

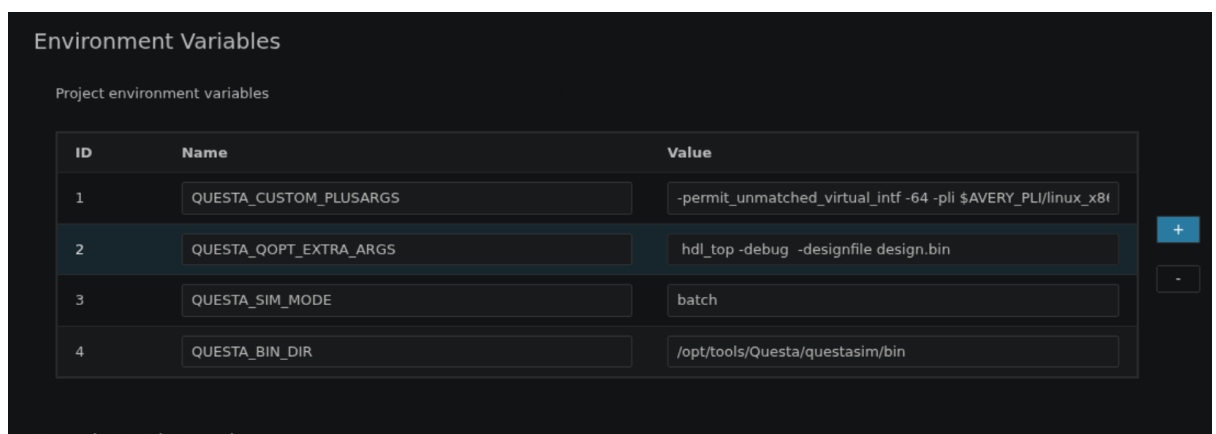
a. Click on “Project Attributes”



b. In the QOPT_EXTRA_ARGS, add hdl_top



11. Change simulation mode from “batch” to “gui”



12. Click on “SAVE”

13. Now RMB on the hvl_top and run simulation using custom plugin

Expected output:

You should see that 5 packets are checked and passed.

But you will see the following UVM_Errors

```
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 220
# UVM_WARNING : 0
# UVM_ERROR : 201
# UVM_FATAL : 0
# ** Report counts by id
# [AAXI4STREAM/SEQ_LIB] 5
# [AAXI4_ERROR] 201
# [AAXI4_TUP] 0
# [AAXI4STREAM/SEQ_LIB] 5
# [CONFIGURATION SETTINGS] 2
# [UVM] 197
# [Questa UVM] 2
# [MNTST] 1
# [TEST_DONE] 1
# [top.test_base] 1
# ** Note: $finish : /u/release/questa/2026.1.1/questasim/linux_x86_64/verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 50655 ns Iteration: 59 Instance: hvl_top
# Stopped at /u/release/questa/2026.1.1/questasim/linux_x86_64/verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
#
# 16:27:55: Loading waveform header...
# 16:27:55: Loaded waveform header
#
# quit
#
# run -all
# Simulation has finished - cannot run or step (this can be disabled in LiveSim prefs).
```

14. Open <task_directory>/top_dir/config_policies for both master and slave and add this **cfg.set_config_int("enable_txn_recording",1);** to enable GUI debugging for the next step.

```
class aaxi4_stream_slave_0_config_policy;
    static function void configure
    (
        input aaxi4_stream_slave_0_cfg_t cfg
    );
        //
        // Agent setup configurations:
        //
        cfg.set_config_int("agent_type",aaxi_pkg::AAXI_SLAVE);
        cfg.set_config_int("is_active",1);
        //
        // BFM setup configurations:
        //
        cfg.set_config_int("stream_if_type",aaxi_pkg::AAXI4STREAM);
        //
        // BFM setup configurations at default value:
        //cfg.set_config_int("tuser_width",AAXI_USER_BYTE_WIDTH*data_bus_bytes);
        // // //----- AXI4 Stream BFM related configurations -----
        cfg.set_config_int("tready_default",1);
    endfunction
endclass
```

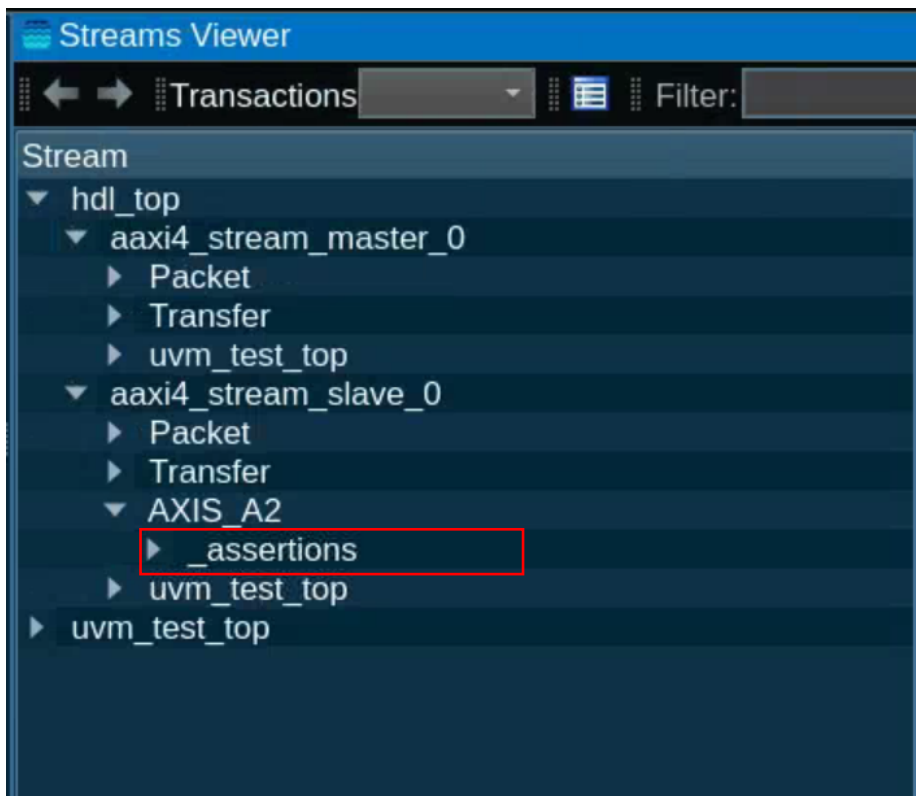
Debugging Protocol Failures using Questa One Sim Debug

First, you need to run the simulation for the UVM testbench simulation created in the previous step in GUI mode to launch the debugger.

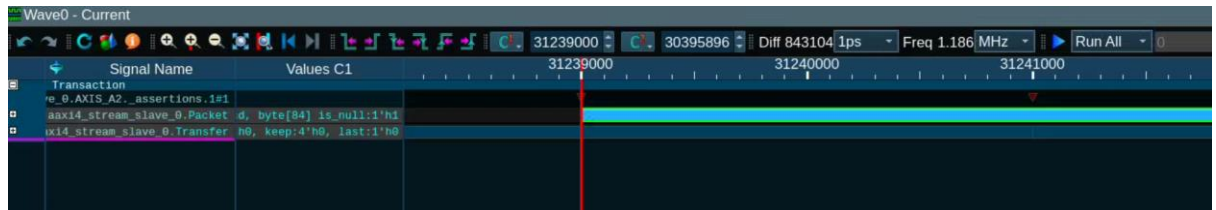
1. After the simulation run is done, you'll notice some UVM_ERRORS appearing coming from the VIP. These are protocol violations which can be debugged.

```
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 220
# UVM_WARNING : 0
# UVM_ERROR : 201
# UVM_FATAL : 0
# ** Report counts by id
# [AAXISTREAM/SEQ_LIB] 5
# [AAXI_ERROR] 201
# [AAXI_INF] 6
# [AXI4STREAM/SEQ_LIB] 5
# [CONFIGURATION SETTINGS] 2
# [FIR_SB] 197
```

2. To start debugging, go to Streams Viewer, from View → Streams Viewer or Ctrl+Shift+5.
3. Start expanding the hierarchy until reaching the assertion AAXI_ERROR that was firing from the VIP agent. Start from hdl_top.



4. To view on waveform, right click `_assertions`, choose Add to wave → Failing Assertions
5. Zoom in on one of the fired assertions, and right click, choose View in Composition Viewer, this will give you description about the failed assertions.
6. Add the slave packets and transfers to the Waveform and Composition Viewer, similar to how it was done for the failing assertion or by dragging and dropping from the Streams Viewer to the Wave window. By clicking on the plus icon next to the transaction name, you'll be able to view the transfers in a protocol format, showing data and different AXI4-stream transfer attributes like id, strb and last.

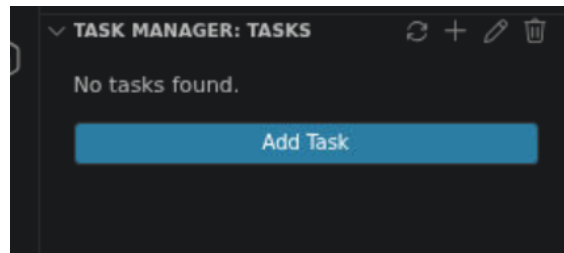


7. You'll see an assertion is fired at the end of each transfer, right click on one of the transfers and select Find → Signal Activity. Signals related to that transaction will be dumped below the transactions in the waveform. You can view the signal view and navigate using the cursor for further debug to see which signal is causing the assertions to fire.
8. Make the necessary changes to the UVM testbench `hdl_top` to stop these assertions from firing.
9. Run again the simulation, make sure that `UVM_ERRORS` count is now 0.

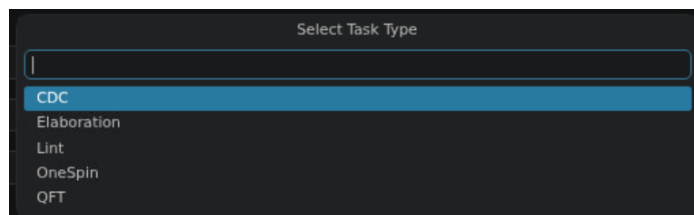
```
# UVM_INFO fir_scoreboard.svh(230) @ 2050655000: uvm_test_top.env.fir_sb [FIR_SB] SCOREBOARD PASSED - 5 packet(s) checked, 0 errors
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 442
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
```

BONUS: Clean Lint

1. In the Task Manager click “Add Task”



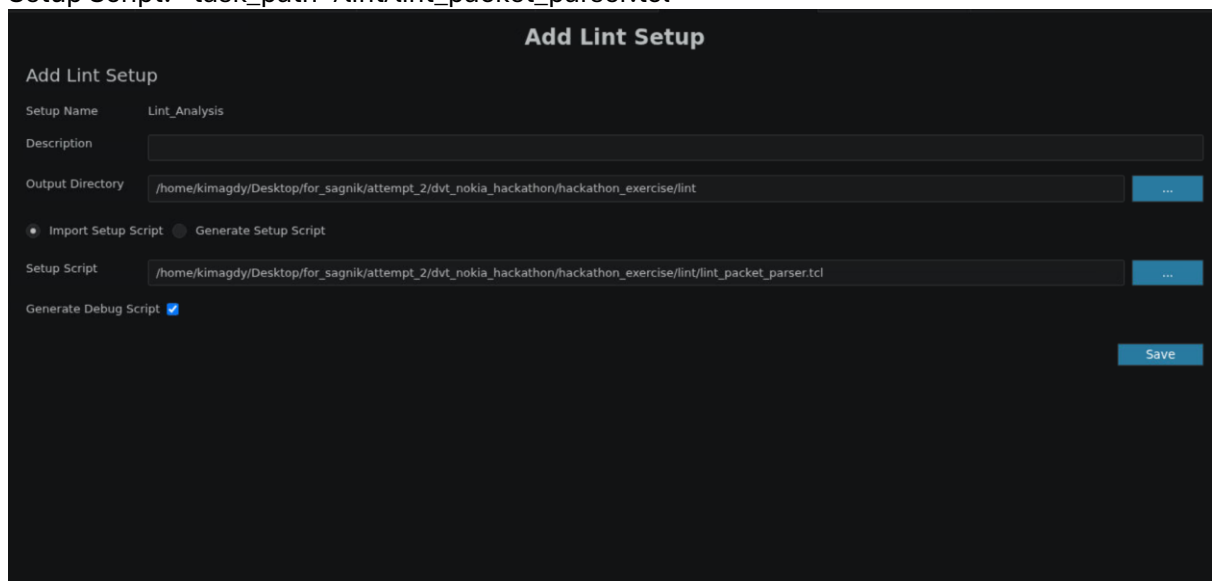
2. From the popping selection menu choose “Lint”



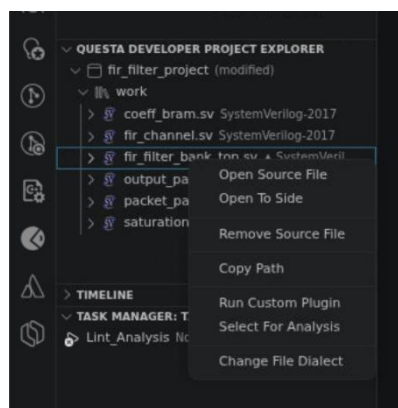
3. Setup Lint Task as follows and click on “Save”

Output Directory: <task_path>/lint

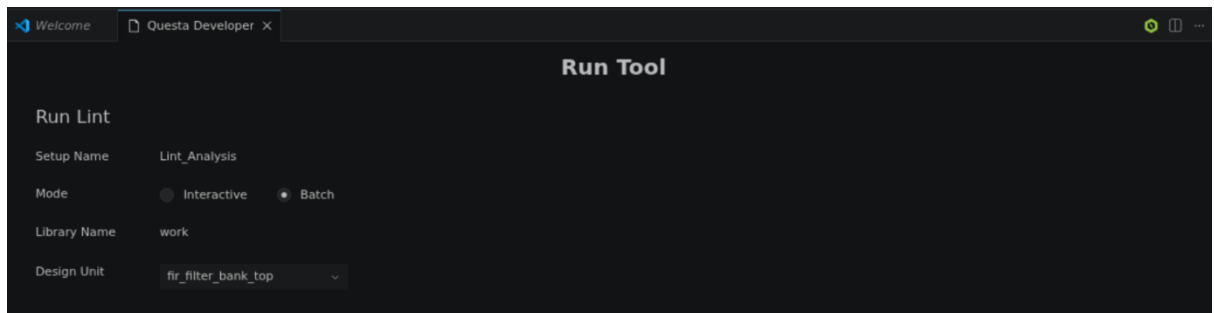
Setup Script: <task_path>/lint/lint_packet_parser.tcl



4. To run Lint on specific top module/module, RMB on “fir_filter_top.sv” from “Questa Developer Project Explorer” and click on “Select For Analysis”



- Click on run task from the Task manager and choose the design unit and click on “Run Lint”



- You will find 21 Errors, clean these errors the fix(s) could be RTL changes or waiving.
- To debug the results, click on the Task debug icon and choose integrated Lint

